Machine Learning Innovations in Code Generation: A Systematic Literature Review of Methods, Challenges and Directions

1st Mardi Siswo Utomo
Dept. of Information Technology and Industry
Universitas Stikubank
Semarang, Indonesia
email: mardi@edu.stikubank.ac.id
ORCID: 0009-0008-8972-908X

2nd Ema Utami
Dept. of Informatics Doctorate
Universitas Amikom Yogyakarta
Yogyakarta, Indonesia
email: ema.u@amikom.ac.id
ORCID: 0000-0002-8237-8693

3rd Kusrini
Dept. of Informatics Doctorate
Universitas Amikom Yogyakarta
Yogyakarta, Indonesia
email: kusrini@amikom.ac.id
ORCID: 0000-0001-9573-3909

4th Arief Setyanto
Dept. of Informatics Doctorate
Universitas Amikom Yogyakarta
Yogyakarta, Indonesia
email: arief_s @amikom.ac.id
ORCID: 0000-0003-0721-3941

2024 International Conference on Information Technology and Computing (ICITCOM) | 979-8-3503-7983-9/24/$31.00 ©2024 IEEE | DOI: 10.1109/ICITCOM62788.2024.10762291

***Abstract*— This paper provides a comprehensive review of recent advances in machine learning (ML) technologies applied to code generation, with a focus on literature published between 2022 and 2024. We collect and analyze 36 studies from four major publication databases: ACM, IEEE Xplore, Springer, and Scopus, highlighting the transformative impact of transformer-based models in this domain.**

Significant findings indicate that transformer-based models and graph neural networks (GNNs) are the best ways to understand and generate code. Exact match accuracy, BLEU, ROUGE, execution verification, and human studies are some of the evaluation methods that now provide comprehensive performance assessments. Major challenges identified include a lack of high-quality training data, high computational resource requirements, and security concerns. Recent trends emphasize the use of diverse neural network architectures, industrial project datasets, and novel evaluation methods to improve model robustness and applicability. Applications cover a wide range of paradigms, including code-to-description, description-to-code, and code-to-code, with tasks such as automated documentation and code optimization. Dataset innovations include the use of open-source repositories, competition datasets, and synthetic data to improve code structural and contextual understanding. This paper seeks to guide future research and foster additional innovations in the field by providing a thorough examination of both progress and challenges in ML-powered code generation.

Keywords—Code Generator, Machine Learning, Transformer, Automatic programming, natural language processing

I. INTRODUCTION

The software development process is a time-consuming and complex process [1], which requires technological breakthroughs in software development to accelerate the process. Code generation is one technology that can accelerate software development, it's has the ability to speed up the code writing process [2]..

Code generator technology has been in development for a long time, dating back to the 1970s [3], [4]. Following the introduction of transformer architecture [5], there has been an increasing amount of research on code generators. This trend is in line with Gartner's findings of a significant increase in demand for no-code and low-code applications since 2020 [6], where code generators are the key component of low-code and no-code applications.

The aim of the literature study is to conduct a systematic review and analysis of the literature on code generator studies using machine learning (ML) methods. We will review studies that employ ML methods and interact directly with the source code. This review covers a wide range of topics, including ML methods and models, evaluation methods, types of applications and paradigms, and datasets used.

To provide a comprehensive view of the latest innovations in automatic code generation using ML, we formulated six research questions (RQs) as follows:

1. RQ1: What are the latest machine learning methods used in automatic code generation research?
2. RQ2: How is the performance of machine learning models in code generation tasks evaluated?
3. RQ3: What are the main challenges in using machine learning models for code generation?
4. RQ4: What are the recent trends in the application of machine learning methods for code generation?
5. RQ5: What are the applications and paradigms of code generation that recent literature has explored?
6. RQ6: What are the innovations in datasets used to train and evaluate machine learning models for code generation?

We hope that this review will provide a comprehensive and in-depth examination of the ML methodology used in the field of automated software generation. Furthermore, we anticipate that this paper will expedite our understanding of the latest ML-based code generator research topics and keep us abreast of advancements in the field. Studies that have successfully improved model capabilities when applied to code generation tasks have revealed the latest advances in ML, with transformer-based models dominating recent research.

II. THE REVIEW

A. Review Procedure

This paper follows the Preferred Reporting Items for Systematic Reviews and Meta-Analyses (PRISMA) guidelines [7], [8]. PRISMA guidelines will ensure the code generation review has a well-structured and replicable methodology. To collate and analyze findings on ML innovations in code generation, the review process includes

Authorized licensed use limited to: UNIVERSITAS GADJAH MADA. Downloaded on January 22,2025 at 06:09:04 UTC from IEEE Xplore. Restrictions apply.

a detailed search strategy, eligibility criteria, and a data synthesis approach.

We conducted literature identification in four publication databases: 1) ACM Digital Library, 2) IEEE Xplore, 3) Springer, and 4) Scopus. Search criteria will be based on the following four inclusion criteria: studies employing ML or deep learning for code generation, summarization, suggestion, and completion; research on code generators across various applications and programming languages; articles published in the last two years; publications outside of peer-reviewed journals or book chapters.

The next step is to determine search terms based on the four criteria for inclusion. When applying the inclusion criteria to the search terms, we use the following search terms: "deep learning" OR "machine learning" OR "transformer" AND "code", and "generator" OR "generation" OR "summarization" OR "completion" OR "suggestion". Initially, we apply the search terms to all metadata and then filter only documents whose titles and abstracts contain keywords related to machine learning or deep learning for code generation, summarization, suggestion, and completion.

The screening process continued by removing duplicate documents that exist in more than one database and discarding documents that meet the four exclusion criteria. We have used four exclusion criteria: non-English articles without an official translation; studies available only in abstract or summary form; research lacking peer review or coming from unverifiable sources; and studies not in the Q1-Q4 JCR rank.

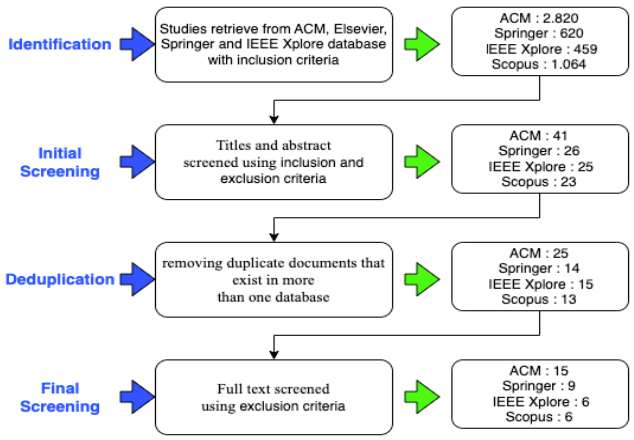


Fig. 1. Document Screening Process

A manual peer review conducted the final screening, confirming the relevance and alignment of the studies with the review's thematic scope. This comprehensive assessment ensured the inclusion of high-quality studies. Figure 1 provides a clearer view of the document selection process.

B. Machine Learning Methods

Research on code generation using machine learning is divided into three categories: model-based algorithms, learning-based approaches, and hybrid engineering, as seen in Figure 2. Many studies focus on modeling-based algorithms, such as transformers and graph neural networks (GNN) [9] [10]. Models such as CSGVD [11] use GNN to combine sequential and graphical information, resulting in a more structured approach to code comprehension and generation. Feng et al. [12] use complex neural networks to generate end-to-end code, which adds to the category's depth.

The learning-based approach employs pre-training models as well as reinforcement learning. Soliman et al.[13] and Li et al. [14] show how pre-training a large corpus can improve code generation in a broader context. Gou et al. [15] use reinforcement learning to optimize code in security and stability control systems, demonstrating the models' flexibility and adaptability under real-world conditions. The challenges of this approach include the requirement for large and diverse training data, as well as intensive computing.

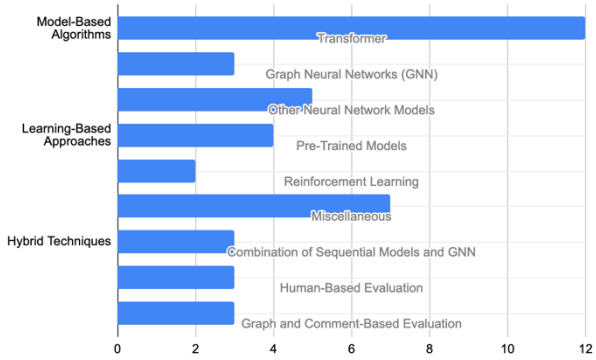


Fig. 2. Distribution of methods

Hybrid engineering combines a variety of methods to improve model performance and efficiency. Tang et al. [11] and MarianCG [16] Demonstrate that combining sequential models and GNN leads to a better understanding of code structure and context. Yang et al. [17] perform human-based evaluation, which provides richer and more contextual feedback, whereas Kuang et al. [18] use graphic-based assessment and code commentary to help understand the relationship between code components. This approach's most notable innovation is its ability to provide a more holistic and comprehensive evaluation, thereby improving the quality and relevance of code generated for practical applications. The challenges of hybrid engineering include integrating various evaluation methods and ensuring that models can function effectively in complex real-world scenarios.

C. Machine Learning Model

As shown in Table 1, we have classified machine learning models in software engineering into five primary categories: Recurrent Neural Networks (RNN), Transformers, Convolutional Neural Networks (CNN), Other Neural Networks, and miscellaneous. We further subdivide each of these categories into subcategories to provide a more specific representation of the applied application and architecture.

In recent years, research into code generation using machine learning (ML) has advanced rapidly, with a variety of neural network models driving innovation in this area. Transformer models, such as those used in ExploitGen for template exploit code augmentation [19] and SeTransformer

Authorized licensed use limited to: UNIVERSITAS GADJAH MADA. Downloaded on January 22,2025 at 06:09:04 UTC from IEEE Xplore. Restrictions apply.

for semantic code parsing [20], have demonstrated a deep understanding of programming language context and structure.

Despite numerous successes, implementing the ML model for code generation remains a significant challenge. One of the primary constraints is the scarcity of high-quality training data, which is required for training models to generalize well. Furthermore, the Transformer model's complexity and scale necessitate massive computing resources, which may impede its development and implementation in real-world settings [21]. Security risks and information leaks are also major concerns, particularly in industrial applications that prioritize security, as noted by Gou et al. in their work on code generation for security and stability control [15]. Furthermore, understanding code semantics in depth remains a significant challenge, despite attempts by models.

TABLE 1 MACHINE LEARNING MODEL STUDIED IN RESEARCH ARTICLES

ML Category	Sub Category	Studies
RNN	Standard RNNs	[22], [23]
	LSTM Variants	[9], [11], [13], [18], [21], [24], [25], [26], [27], [28], [29], [30]
Transformer	Transformer Models	[10], [14], [15], [16], [17], [19], [20], [31], [32], [33], [34], [35], [36], [37], [38], [39]
CNN	Image-related CNNs	[40]
Other Neural Network	Generative Models	[20], [35]
	Other Various Neural Network Models	[41], [42], [43], [44]
Miscellaneous	Traditional ML Techniques & Symbolic Learning	[12]

Recent research trends indicate an increased integration of various types of neural networks in order to capitalize on the strengths of each architecture [11]. The combination of various generative neural network models and traditional symbolic learning techniques aids in many aspects of code generation. Future research will push the limits of efficiency and accuracy in ML models for code generation, with an emphasis on model optimization and training data quality.

D. Evaluation Methods

Researchers have identified a diverse array of evaluation methods in the systematic review of methodologies used across various studies on code generation with machine learning, highlighting the multifaceted approach they take to validate their models and results. Our categorization of these methods is shown in Table 2. We categorize the evaluation methods into a streamlined set of categories that not only highlights commonalities but also provides insights into the domain's prevailing focus areas.

Accuracy-based metrics, such as exact match and token-level accuracy, are popular evaluation methods. Some researcher uses exact match accuracy to assess the model's success in generating code that corresponds to the desired solution [9]. Furthermore, the READSUM model [37] adapts BLEU and ROUGE-based metrics, which are commonly used in natural language processing (NLP), to assess the quality of code generation by comparing the generated code and reference code.

In addition to accuracy-based metrics, we use execution and runtime errors to assess the effectiveness of models in generating error-free executable code. Some models use this method to ensure not only the syntax correctness of the generated exploit code, but also its correct and effective execution [19]. Semantic evaluation, which includes syntax and semantic analysis of the code generated, is also required to ensure that the code is not only technically correct but also consistent with its intended purpose.

TABLE 2 EVALUATION METHODS STUDIED IN RESEARCH ARTICLES

Evaluation Method	Studies
Core Language Metric	[9], [13], [14], [16], [17], [18], [19], [21], [22], [23], [24], [25], [28], [33], [34], [37], [38], [42]
Specific Dataset Evaluation	[10], [11]
Execution Verification and Comparative Analysis	[12], [15], [43]
Advanced Metrics and Analysis (Precision, Recall)	[26], [30], [31], [32], [35], [36]
Application-Specific and Platform-Specific Studies	[29], [39], [40], [41], [44]
Code Analysis Assessment	[20], [27]

The most recent innovations in code generation evaluation methods indicate a shift toward more comprehensive and contextual evaluation. The use of graphical evaluation and code comments. Kuang et al. [18] created a model that uses GNN-based evaluation to assess its understanding of complex code structure and context. This evaluation includes a graphical analysis to help you understand the relationship between code components. Furthermore, Yang et al.'s research [17] uses the human studies evaluation method to assess the quality and utility of code from the perspective of a human programmer, providing valuable insights into code comprehension and practical application. Gou et al. [15] use data sets from industrial projects to test models under more difficult and realistic conditions, resulting in another innovation in system security and stability controls. By using more holistic and contextual evaluation methods, ML code generation research can produce more robust and relevant models for practical applications.

E. Application Types and Paradigms

Mapping application types and paradigms is critical for driving innovation and further development in the field of code generation. To create more comprehensive and useful models, researchers must seek out the most recent ideas and trends in each field. Such as used human studies to assess code utility [17], combined graphic-based evaluation and code commentary [18]. This innovation not only improves the model's quality and relevance, but it also creates new opportunities for practical software applications.

Table 3 presents a structured distribution of the application types among different computational paradigms, covering 37 studies in total. These paradigms are defined as CTC (Code-to-Code), CTD (Code-to-Description), and DTC, as well as DTC-CTD (Description-to-Code and Code-to-Description) combinations. Each paradigm is associated with code-related tasks such as code completion, code generation, and code summary.

Authorized licensed use limited to: UNIVERSITAS GADJAH MADA. Downloaded on January 22,2025 at 06:09:04 UTC from IEEE Xplore. Restrictions apply.

The research on machine learning (ML) code generation covers a wide range of applications and different code generation paradigms. The review paper's analysis categorizes the resulting applications into three main paradigms: code to description, description to code, and code to code. For instance, READSUM [37] model operates within the code-to-description paradigm, analyzing source code to generate meaningful descriptions or summaries. This paradigm is useful for automated documentation and code comprehension, helping developers understand existing code functions and structures. Yang et al.'s [17] research also focused on the importance of naming good methods for aiding code understanding and integrating semantic evaluation into code generation processes.

TABLE 3 PARADIGM - APPLICATION TYPES AND PARADIGMS STUDIED IN RESEARCH ARTICLES

Paradigm	Application Types	Freq.	Studies
CTC	Code Completion	1	[17]
	Code Generation	2	[11], [14], [26]
	Code Summarization	1	[29]
Subtotal		4	
CTD	Code Generation	1	[15]
	Code Summarization	5	[9], [21], [24], [37]
Subtotal		6	
DTC	Code Summarization	1	[44]
	Code Completion	1	[25]
	Code Generation	14	[10], [12], [13], [16], [19], [20], [27], [34], [35], [36], [38], [39], [41], [43]
Subtotal		17	
DTC - CTD	Code Completion	4	[22], [28], [30], [31]
	Code Generation	4	[23], [33], [40], [42]
	Code Summarization	2	[18]
Subtotal		10	
Total		**36**	

The description-to-code code-to-description Combined Paradigm: these paradigms reflect diverse approaches in the field of software engineering, where the integration of natural language processing with analysis and code generation stands out as a significant trend. This table not only describes current research focus areas but also potential cross-disciplinary methods that combine code understanding with automatic generation tasks.

We found that the latest innovation in evaluation methods for code generation indicates a trend towards more holistic and contextual evaluation. One of the prominent innovations is the use of human studies to assess the usefulness and quality of the code produced. For instance, Yang et al. [17] utilized human programmers' evaluations to gauge the model's ability to comprehend and apply the code in real-world scenarios. The Kuang et al. model [18], [20] also includes graphical evaluations and code commentaries, which help test how well people understand models with more complicated code structures and contexts. This approach shows how a more thorough and contextual evaluation can improve the quality and relevance of code generation models in practical applications. By adopting more holistic and contextualized evaluation methods, research in the field of ML code generation can produce a more robust and relevant model for practical applications.

F. *Datasets*

Table 4, we catalog in detail the datasets used in various studies in the creation of ML code, revealing different data sources. These are public data sets that each serve unique purposes and challenges in research.

The quality and variation of the datasets used to train and evaluate models are critical in code generation research using machine learning (ML). Many researchers use datasets from open-source code repositories like GitHub, which offer code in a wide range of languages and complexities. For example, Codeforces competition dataset that contains well-structured programming problems and solutions [10]. The DLBT model also employs datasets from an open repository to train Transformers for source code generation [21]. This method enables models to learn from a variety of programming styles, improving generalization.

TABLE 4 DATASETS USED IN RESEARCH ARTICLES

Dataset	Frequency	Studies
GitHub	8	[10], [22], [24], [25], [29], [30], [35], [42]
CodeSearchNet	3	[14], [25], [38]
CodeXGLUE	3	[11], [34], [38]
DJANGO	3	[13], [16], [21]
CoNaLa	2	[13], [16]
Human-Eval	2	[17], [36]
MBPP	2	[17], [36]

However, the use of datasets in code generation research presents some challenges. The lack of high-quality training data is a significant barrier, as the model requires data covering a wide range of code types and usage scenarios to improve generalization. Furthermore, some studies use synthetic datasets to address the lack of real-world data. For example, Auto-Icon+ [12] tests their end-to-end systems using automatically generated datasets, whereas Web Generator [20] tests with a synthetic dataset. Although synthetic datasets help to control experimental variables, they may not fully represent the complexity of real-world code, limiting models' capabilities in practical situations.

The most recent advancements in the use of datasets for code generation demonstrate a trend toward integrating various types of data to improve model performance. The CSGVD model combines sequential and graphical data using graph neural networks (GNN), demonstrating significant potential for improving structural and contextual understanding of code [11]. Furthermore, Chen et al. demonstrate how models can use non-traditional data for code generation tasks by training them with data from a graphical user interface (GUI) [27]. Other innovations include using datasets from industrial projects to ensure that models can operate on a large scale and in real-world conditions, as demonstrated by Gou et al. in their control of system security and stability [15].

III. DISCUSSION

The landscape of machine learning (ML) innovations in code generation is diverse and rapidly evolving, as evidenced by the analysis of 36 papers. These studies employ a variety of methods, models, applications, paradigms, and evaluation techniques, reflecting the multi-faceted nature of the field.

Methods and Models: The reviewed papers demonstrate the widespread use of model-based algorithms, specifically transformers and graph neural networks (GNNs).

Authorized licensed use limited to: UNIVERSITAS GADJAH MADA. Downloaded on January 22,2025 at 06:09:04 UTC from IEEE Xplore. Restrictions apply.

Transformers [10], [11], excel in semantic parsing and competitive programming. The GNNs effectively combine sequential and graphical data to enhance code generation. Additionally, learning-based approaches, including pre-trained models (e.g., BERT, GPT) and reinforcement learning, demonstrate significant potential. Some papers are also highlight these models' adaptability and context-aware generation capabilities [13], [15]. Hybrid techniques that integrate sequential models with GNNs and human-based evaluations provide a more holistic approach to code generation.

Applications and Paradigms: ML's applications in code generation span a variety of paradigms, including code to description, description to code, and code to code transformations. Code-to-description paradigms, such as READSUM, focus on generating meaningful summaries from source code, aiding in documentation and comprehension. Description-to-code approaches, translate natural language descriptions or graphical interfaces into executable code, streamlining the software development process. Code-to-code transformations, involve optimizing or refactoring existing code, enhancing efficiency and maintainability.

Evaluation Methods: To evaluate the effectiveness of these models, various metrics and techniques are used. Quantitative assessments commonly use exact match accuracy, BLEU, and ROUGE scores. Some papers employ these metrics to measure the syntactic and semantic accuracy of generated code [18], [24], [37]. Execution-based evaluation, ensures the generated code is not only correct but also functional [19]. Human studies, provide qualitative insights into the usability and comprehensibility of the generated code [17]. Graph and comment-based evaluations, help assess the model's understanding of complex code structures and interrelations [18].

IV. Conclusion

This systematic literature review examines the latest research in machine learning technology (ML) applied to code generation, focusing on studies published between 2022 and 2024. We concluded the current state and future direction of this rapidly growing field by analyzing 36 studies from a major publication database. Here are a few things we can conclude.

Transformer-based models and graph neural networks (GNNs) have dominated recent research in code generation domains. Transformers and GNN outperform traditional models like RNN and CNN in terms of sequential and graphical data processing, allowing for a more in-depth and accurate understanding of code. Furthermore, pre-training models like BERT and GPT, as well as reinforcement learning, show great promise for improving code generation capabilities. This method outperforms traditional approaches because it can better handle the complexity of programming languages.

The combination of sequential models, GNNs, and human evaluation provides a comprehensive approach that improves the quality and relevance of the code generated. In comparison to traditional evaluation methods, this technique improves understanding of code structure and context. Evaluation using accuracy metrics, execution verification, semantic analysis, and human evaluation provides a comprehensive picture of the model's performance. These evaluation methods go beyond simply using accurate metrics, ensuring that the code is not only syntax correct but also functional and understandable.

The study examines a wide range of application paradigms, including code-to-description, description-to-code, and code-to-code, to demonstrate the widespread use of machine learning in code generation. Compared to traditional methods, this approach is more effective for a variety of tasks, including automated documentation and code optimization. Using data sets from open-source repositories like GitHub, CodeSearchNet, and CodeXGLUE, as well as synthetic datasets, helps to overcome the problem of a lack of high-quality training data. It addressed the shortcomings of previous datasets, which tended to overlook a wide range of real-world scenarios, thereby improving the structural and contextual understanding of code.

this study has made an important contribution to understanding the most recent advances in code generation with machine learning. Determine which models and methods, such as transformers, GNN, and hybrid techniques, outperform traditional methods in terms of comprehension and coding. The study also emphasizes the importance of employing comprehensive evaluation methods, such as human evaluation, to assess the quality and utility of the code produced.

This study has important implications for the future development of code generation technology. The use of transformers and GNN in code generation has demonstrated that this model can provide a deeper and more accurate understanding of code, implying increased software development efficiency and quality. Furthermore, the use of hybrid techniques that combine various evaluation and learning methods, including human evaluation, provides a more comprehensive and realistic approach to evaluating model performance.

Future research could focus on the following key areas: 1) High-Quality Training Data Development: To address the issue of a lack of high-quality training data, develop and use more diverse and representative datasets of real-world scenarios. 2) Computing Resource Optimization: Reduces the need for large computing resources by utilizing model optimization techniques and more efficient computing methods. 3) Security and Privacy: Improving models to address security and privacy concerns, particularly in industrial applications that are highly vulnerable to data leaks. 4) Create more comprehensive evaluation methods that include both qualitative and quantitative evaluation, as well as expanded human evaluation, to gain a better understanding of the usefulness of the code generated. 5) Integration of Sequential and GNN Models: Improves the integration of sequential models and GNNs to maximize the ability to understand structural and contextual codes. As a result, we expect that this research will accelerate understanding and innovation in ML-based code generation, keep this field at the forefront of technological development, and show significant improvements over previous methods.

References

[1] E. Dehaerne, B. Dey, S. Halder, S. De Gendt, and W. Meert, “Code Generation Using Machine Learning: A Systematic Review,” *IEEE Access*, vol. 10, pp. 82434–82455, 2022, doi: 10.1109/ACCESS.2022.3196347.

[2] C. W. Fraser, D. R. Hanson, and T. A. Proebsting, “Engineering a simple, efficient code-generator generator,” *ACM Lett. Program. Lang. Syst.*, 2002, doi: 10.1145/151640.151642.

Authorized licensed use limited to: UNIVERSITAS GADJAH MADA. Downloaded on January 22,2025 at 06:09:04 UTC from IEEE Xplore. Restrictions apply.

[3] C. W. Fraser, "Automatic generation of code generators.," phd, Yale University, USA, 1977.

[4] J. Herrington, "Code Generation in Action," Manning Publications. Accessed: May 10, 2024. [Online]. Available: https://www.manning.com/books/code-generation-in-action

[5] A. Vaswani *et al.*, "Attention is All you Need," in *Advances in Neural Information Processing Systems*, Curran Associates, Inc., 2017. Accessed: Jul. 12, 2023. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html

[6] "Gartner Forecasts Worldwide Low-Code Development Technologies Market to Grow 20% in 2023," Gartner. Accessed: Feb. 18, 2024. [Online]. Available: https://www.gartner.com/en/newsroom/press-releases/2022-12-13-gartner-forecasts-worldwide-low-code-development-technologies-market-to-grow-20-percent-in-2023

[7] M. J. Page *et al.*, "The PRISMA 2020 statement: an updated guideline for reporting systematic reviews," *BMJ*, vol. 372, p. n71, Mar. 2021, doi: 10.1136/bmj.n71.

[8] A. Liberati *et al.*, "The PRISMA Statement for Reporting Systematic Reviews and Meta-Analyses of Studies That Evaluate Health Care Interventions: Explanation and Elaboration," *PLOS Med.*, vol. 6, no. 7, p. e1000100, Jul. 2009, doi: 10.1371/journal.pmed.1000100.

[9] Z. Li *et al.*, "SeTransformer: A Transformer-Based Code Semantic Parser for Code Comment Generation," *IEEE Trans. Reliab.*, vol. 72, no. 1, pp. 258–273, Mar. 2023, doi: 10.1109/TR.2022.3154773.

[10] Y. Li, "Competition-level code generation with AlphaCode," *Science*, vol. 378, no. 6624. pp. 1092–1097, 2022. doi: 10.1126/science.abq1158.

[11] W. Tang, M. Tang, M. Ban, Z. Zhao, and M. Feng, "CSGVD: A deep learning approach combining sequence and graph embedding for source code vulnerability detection," *J. Syst. Softw.*, vol. 199, p. 111623, May 2023, doi: 10.1016/j.jss.2023.111623.

[12] S. Feng, M. Jiang, T. Zhou, Y. Zhen, and C. Chen, "Auto-Icon+: An Automated End-to-End Code Generation Tool for Icon Designs in UI Development," *ACM Trans. Interact. Intell. Syst.*, vol. 12, no. 4, p. 36:1-36:26, Nov. 2022, doi: 10.1145/3531065.

[13] A. Soliman, "Leveraging pre-trained language models for code generation," *Complex and Intelligent Systems*. 2024. doi: 10.1007/s40747-024-01373-8.

[14] J. Li *et al.*, "CodeEditor: Learning to Edit Source Code with Pre-trained Models," *ACM Trans. Softw. Eng. Methodol.*, vol. 32, no. 6, p. 143:1-143:22, Sep. 2023, doi: 10.1145/3597207.

[15] Q. Gou, Y. Dong, and B. Shen, "Code generation for Security and Stability Control System based on extended reactive component," *J. Syst. Archit.*, vol. 148, p. 103069, Mar. 2024, doi: 10.1016/j.sysarc.2024.103069.

[16] A. S. Soliman, "MarianCG: a code generation transformer model inspired by machine translation," *Journal of Engineering and Applied Science*, vol. 69, no. 1. 2022. doi: 10.1186/s44147-022-00159-4.

[17] G. Yang, Y. Zhou, W. Yang, T. Yue, X. Chen, and T. Chen, "How Important Are Good Method Names in Neural Code Generation? A Model Robustness Perspective," *ACM Trans. Softw. Eng. Methodol.*, vol. 33, no. 3, p. 60:1-60:35, Mar. 2024, doi: 10.1145/3630010.

[18] L. Kuang, C. Zhou, and X. Yang, "Code comment generation based on graph neural network enhanced transformer model for code understanding in open-source software ecosystems," *Autom. Softw. Eng.*, vol. 29, no. 2, p. 43, Jun. 2022, doi: 10.1007/s10515-022-00341-1.

[19] G. Yang, Y. Zhou, X. Chen, X. Zhang, T. Han, and T. Chen, "ExploitGen: Template-augmented exploit code generation based on CodeBERT," *J. Syst. Softw.*, vol. 197, p. 111577, Mar. 2023, doi: 10.1016/j.jss.2022.111577.

[20] A. Soto, H. Mora, and J. A. Riascos, "Web Generator: An open-source software for synthetic web-based user interface dataset generation," *SoftwareX*, vol. 17, p. 100985, Jan. 2022, doi: 10.1016/j.softx.2022.100985.

[21] W. Gad, "DLBT: Deep learning-based transformer to generate pseudo-code from source code," *Computers, Materials and Continua*, vol. 70, no. 2. pp. 3117–3132, 2022. doi: 10.32604/cmc.2022.019884.

[22] F. Liu *et al.*, "Non-Autoregressive Line-Level Code Completion," *ACM Trans. Softw. Eng. Methodol.*, Feb. 2024, doi: 10.1145/3649594.

[23] Y. Xiao, W. Song, J. Qi, B. Viswanath, P. McDaniel, and D. Yao, "Specializing Neural Networks for Cryptographic Code Completion Applications," *IEEE Trans. Softw. Eng.*, vol. 49, no. 06, pp. 3524–3535, Jun. 2023, doi: 10.1109/TSE.2023.3265362.

[24] S. Gao *et al.*, "Code Structure–Guided Transformer for Source Code Summarization," *ACM Trans. Softw. Eng. Methodol.*, vol. 32, no. 1, p. 23:1-23:32, Feb. 2023, doi: 10.1145/3522674.

[25] W. Takerngsaksiri, C. Tantithamthavorn, and Y.-F. Li, "Syntax-aware on-the-fly code completion," *Inf. Softw. Technol.*, vol. 165, no. C, Jan. 2024, doi: 10.1016/j.infsof.2023.107336.

[26] K. Lano, "Code Generation by Example Using Symbolic Machine Learning," *SN Computer Science*, vol. 4, no. 2. 2023. doi: 10.1007/s42979-022-01573-4.

[27] W. Y. Chen, "Code generation from a graphical user interface via attention-based encoder–decoder model," *Multimedia Systems*, vol. 28, no. 1. pp. 121–130, 2022. doi: 10.1007/s00530-021-00804-7.

[28] M. Sharma, T. K. Mishra, and A. Kumar, "Source code auto-completion using various deep learning models under limited computing resources," *Complex Intell. Syst.*, vol. 8, no. 5, pp. 4357–4368, Oct. 2022, doi: 10.1007/s40747-022-00708-7.

[29] B. Pathik and M. Sharma, "Source code change analysis with deep learning based programming model," *Autom. Softw. Eng.*, vol. 29, no. 1, p. 15, Jan. 2022, doi: 10.1007/s10515-021-00305-x.

[30] Z. Alizadehsani, "DCServCG: A data-centric service code generation using deep learning," *Engineering Applications of Artificial Intelligence*, vol. 123. 2023. doi: 10.1016/j.engappai.2023.106304.

[31] Y. Hussain, Z. Huang, Y. Zhou, and S. Wang, "Boosting source code suggestion with self-supervised Transformer Gated Highway," *J. Syst. Softw.*, vol. 196, p. 111553, Feb. 2023, doi: 10.1016/j.jss.2022.111553.

[32] A. Zhang, L. Fang, C. Ge, P. Li, and Z. Liu, "Efficient transformer with code token learner for code clone detection," *J. Syst. Softw.*, vol. 197, p. 111557, Mar. 2023, doi: 10.1016/j.jss.2022.111557.

[33] Y. Liu *et al.*, "Refining ChatGPT-Generated Code: Characterizing and Mitigating Code Quality Issues," *ACM Trans. Softw. Eng. Methodol.*, Jan. 2024, doi: 10.1145/3643674.

[34] S. Tipirneni, "StructCoder: Structure-Aware Transformer for Code Generation," *ACM Transactions on Knowledge Discovery from Data*, vol. 18, no. 3. 2024. doi: 10.1145/3636430.

[35] S. Thakur *et al.*, "VeriGen: A Large Language Model for Verilog Code Generation," *ACM Trans. Des. Autom. Electron. Syst.*, Feb. 2024, doi: 10.1145/3643681.

[36] F. Cassano *et al.*, "MultiPL-E: A Scalable and Polyglot Approach to Benchmarking Neural Code Generation," *IEEE Trans. Softw. Eng.*, vol. 49, no. 07, pp. 3675–3691, Jul. 2023, doi: 10.1109/TSE.2023.3267446.

[37] Y. Choi, C. Na, H. Kim, and J.-H. Lee, "READSUM: Retrieval-Augmented Adaptive Transformer for Source Code Summarization," *IEEE Access*, vol. 11, pp. 51155–51165, 2023, doi: 10.1109/ACCESS.2023.3271992.

[38] Q. Gou, Y. Dong, Y. Wu, and Q. Ke, "RRGcode: Deep hierarchical search-based code generation," *J. Syst. Softw.*, vol. 211, p. 111982, May 2024, doi: 10.1016/j.jss.2024.111982.

[39] M. Li *et al.*, "Towards optimized tensor code generation for deep learning on sunway many-core processor," *Front. Comput. Sci.*, vol. 18, no. 2, p. 182101, Sep. 2023, doi: 10.1007/s11704-022-2440-7.

[40] T. Kaluarachchi, "WebDraw: A machine learning-driven tool for automatic website prototyping," *Science of Computer Programming*, vol. 233. 2024. doi: 10.1016/j.scico.2023.103056.

[41] M. Pavan, E. Ostrovan, A. Caltabiano, and M. Roveri, "TyBox: an automatic design and code-generation toolbox for TinyML incremental on-device learning," *ACM Trans. Embed. Comput. Syst.*, Jun. 2023, doi: 10.1145/3604566.

[42] S. Aşik and U. Yayan, "Generating Python Mutants From Bug Fixes Using Neural Machine Translation," *IEEE Access*, vol. 11, pp. 85678–85693, 2023, doi: 10.1109/ACCESS.2023.3302695.

[43] M. V, T. T. George, and R. K. Harendra, "Enhanced model-driven web application development with code generation using deep learning technique," *Intell. Decis. Technol.*, vol. 18, no. 1, pp. 75–90, Jan. 2024, doi: 10.3233/IDT-220319.

[44] S. Xiao *et al.*, "UI semantic component group detection: Grouping UI elements with similar semantics in mobile graphical user interface," *Displays*, vol. 83, p. 102679, Jul. 2024, doi: 10.1016/j.displa.2024.102679.

Authorized licensed use limited to: UNIVERSITAS GADJAH MADA. Downloaded on January 22,2025 at 06:09:04 UTC from IEEE Xplore. Restrictions apply.